Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Веб-программирование

Лабораторная работа №2

Вариант 888

Выполнил:
Шалабодов Ярослав Дмитриевич, P3210

Проверил:
Дениченко Александр Олегович

Санкт-Петербург
2026

Содержание

1 Задание **2**

2 Исходный код **3**

3 Результат работы программы **3**

4 Дополнительное задание **3**

5 Выводы **4**

1 Задание

Доработать ЛР №1 и разработать FastCGI-сервер на языке Java, определяющий попадание точки на координатной плоскости в заданную область, и создать HTML-страницу, которая формирует данные для отправки их на обработку этому серверу.

Параметр R и координаты точки должны передаваться серверу посредством HTTP-запроса. Сервер должен выполнять валидацию данных и возвращать HTML-страницу с таблицей, содержащей полученные параметры и результат вычислений — факт попадания или непопадания точки в область (допускается в ответе сервера возвращать JSON-строку вместо HTML-страницы). Предыдущие результаты должны сохраняться между запросами и отображаться в таблице.

Кроме того, ответ должен содержать данные о текущем времени и времени работы скрипта.

Комментарии по выполнению ЛР:

- Требуется поднять веб-сервер Apache httpd от лица своего пользователя на Helios (шаблон файла конфигурации доступен для скачивания на se.ifmo).
- Веб-сервер должен заниматься обслуживанием статического контента (HTML, CSS, JS) и перенаправлять запросы за динамическим контентом к FastCGI-серверу.
- FastCGI-сервер требуется реализовать на языке Java (библиотека в виде jar-архива доступна для скачивания на se.ifmo) и поднять также на Helios.
- Путём обращений из JavaScript к FastCGI-серверу требуется показать понимание принципа AJAX.

Разработанная HTML-страница должна удовлетворять следующим требованиям (дополнительным по отношению к ЛР №1, различаются по варианту):

- Данные формы должны передаваться на обработку посредством POST-запроса.

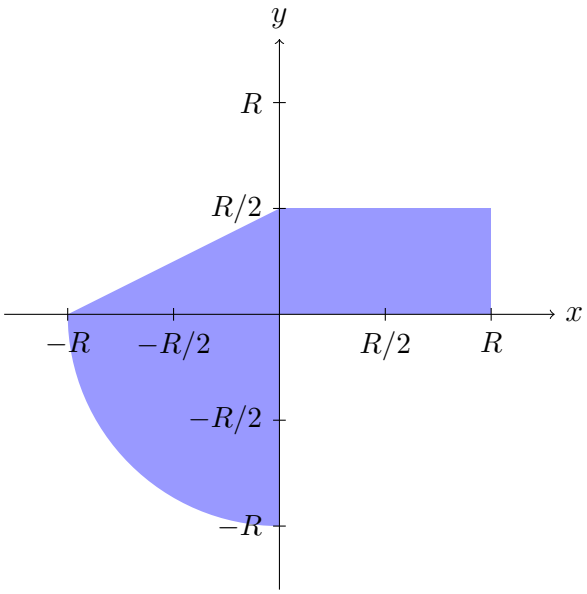


Рис. 1: Область по варианту

2 Исходный код

Ссылка на репо: `https://github.com/1shayd1/ITMO/tree/main/Web/Lab%202`

3 Результат работы программы

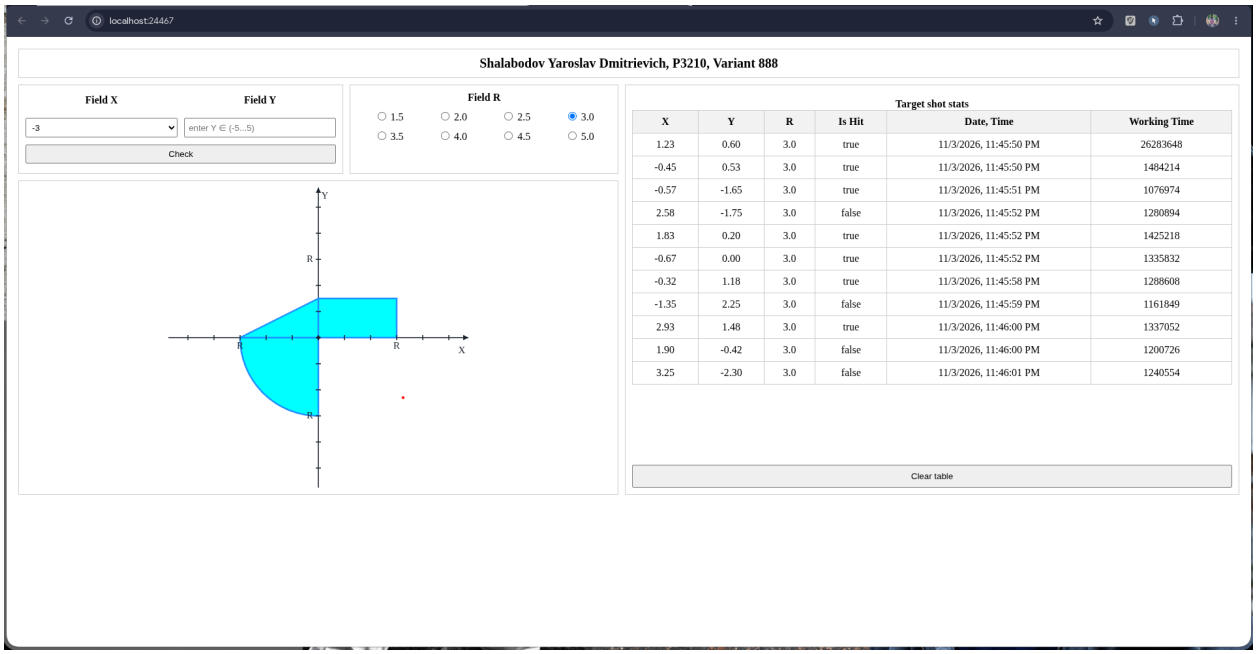


Рис. 2: Внешний вид страницы и таблица результатов

4 Дополнительное задание

Дополнительным заданием было добавить на сервер логирование с трассировкой запросов и подключить готовую систему сбора и просмотра логов.

Для трассировки на клиенте перед каждым запросом генерируется уникальный идентификатор через `crypto.randomUUID()`, который передаётся в заголовке `X-Request-ID`:

Apache передаёт этот заголовок FastCGI-серверу в параметре `HTTP_X_REQUEST_ID`. Сервер берёт из него идентификатор (если заголовка нет, генерирует) и кладёт его в MDC, чтобы он автоматически попадал во все записи лога, относящиеся к этому запросу. После обработки запроса MDC очищается.

Для логирования я использовал SLF4J и Logback. В каждой строке лога указывается уровень, имя сервиса, порт, на котором запущен сервер, и идентификатор запроса. Логи пишутся в файл и отправляются в Grafana Loki через аппендер Loki4j. В Loki имя сервиса, порт и уровень передаются как метки, а traceId — как структурированные метаданные.

Loki и Grafana поднимаются через Docker Compose. В Grafana логи можно смотреть, фильтровать по сервису, порту и уровню и искать все записи по конкретному traceId.

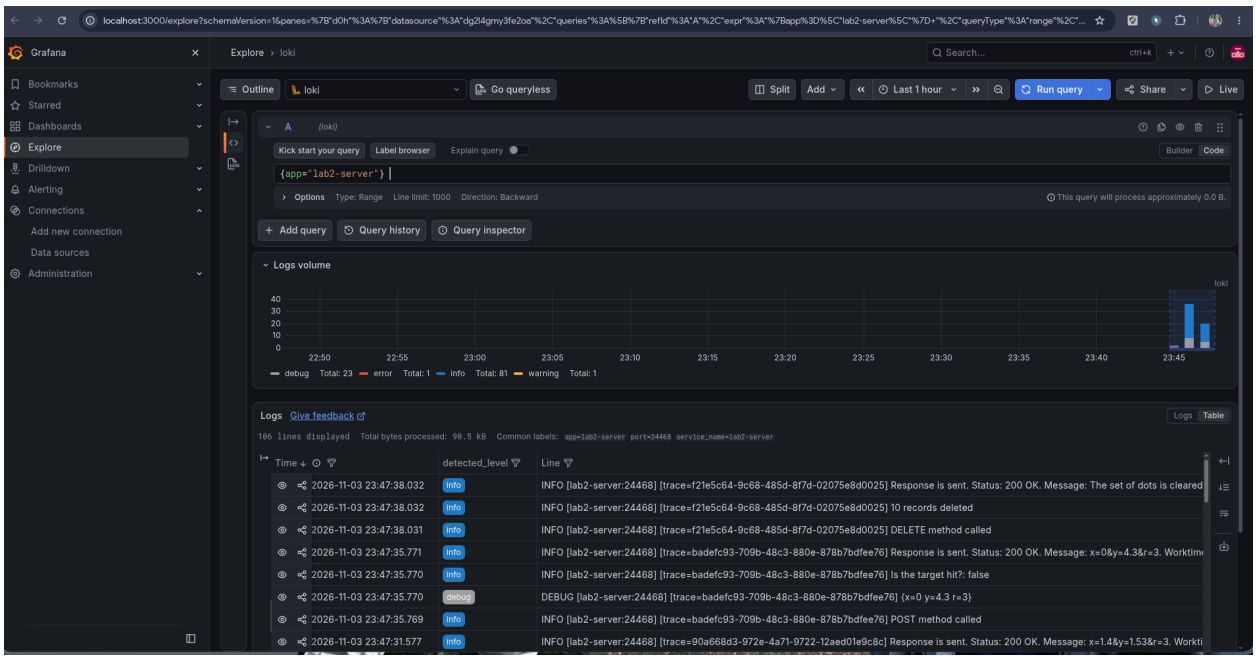


Рис. 3: Просмотр логов в Grafana

5 Выводы

В ходе выполнения лабораторной работы я написал веб-приложение с клиентской и серверной частью. На клиенте форма и холст остались из первой лабы, проверка попадания теперь выполняется на сервере, данные отправляются асинхронно через Fetch API.

Серверную часть я написал на Java с использованием библиотеки FastCGI. Разобрался, чем FastCGI отличается от CGI: сервер запускается один раз и обрабатывает запросы в цикле, а не создаёт новый процесс на каждый запрос. Настроил Apache httpd на гелиосе, чтобы он перенаправлял запросы к FastCGI серверу. Понял, как сервер получает метод запроса и его тело, и как вручную сформировать HTTP-ответ с правильными заголовками. При ошибке сервер возвращает код ошибки (4xx или 5xx) с её описанием, понятным пользователю. Кроме POST сервер поддерживает GET для загрузки истории и DELETE для очистки таблицы.

Во время выполнения дополнительного задании я добавил логирование с трассировкой запросов и подключил Grafana Loki и Loki4j для сбора и просмотра логов. Это необходимо, потому что, когда сервер обрабатывает много запросов, без идентификатора, который показывает, к какому именно запросу относится лог, сложно определить, где выскочила ошибка (если она была). traceId генерируется на клиенте и проходит через весь путь запроса, в консоли браузера можно сразу найти все записи на сервере. Имя сервиса и порт нужны, чтобы различать логи, если запущено несколько экземпляров сервера, а готовая система сбора логов позволяет не читать файл вручную, а удобно искать и фильтровать записи. Также я разобрался, как устроено логирование в Java через SLF4J и Logback, что такое уровни логирования и как настраиваются ап-

пендеры и шаблоны вывода. Узнал, что такое MDC и как с его помощью добавлять контекст запроса ко всем записям лога, не передавая его вручную в каждый вызов. Понял, как HTTP-заголовки запроса передаются FastCGI серверу через параметры. Также научился поднимать Grafana и Loki через Docker Compose.

С выполнением дополнительного задания, мне помогал Claude AI. Также он мне помог разобраться в том, как настроить Apache httpd на гелиосе, поднять FastCGI сервер, грамотно использовать Grafana Loki и Loki4j.